
5 Monte Carlo Methods

In this chapter we consider our first learning methods for estimating value functions and discovering optimal policies. Unlike the previous chapter, here we do not assume complete knowledge of the environment. Monte Carlo methods require only *experience*—sample sequences of states, actions, and rewards from on-line or simulated interaction with an environment. Learning from *on-line* experience is striking because it requires no prior knowledge of the environment’s dynamics, yet can still attain optimal behavior. Learning from *simulated* experience is also powerful. Although a model is required, the model need only generate sample transitions, not the complete probability distributions of all possible transitions that are required by dynamic programming (DP) methods. In surprisingly many cases it is easy to generate experience sampled according to the desired probability distributions, but infeasible to obtain the distributions in explicit form.

Monte Carlo methods are ways of solving the reinforcement learning problem based on averaging sample returns. To ensure that well-defined returns are available, we define Monte Carlo methods only for episodic tasks. That is, we assume experience is divided into episodes, and that all episodes eventually terminate no matter what actions are selected. It is only upon the completion of an episode that value estimates and policies are changed. Monte Carlo methods are thus incremental in an episode-by-episode sense, but not in a step-by-step sense. The term “Monte Carlo” is often used more broadly for any estimation method whose operation involves a significant random component. Here we use it specifically for methods based on averaging complete returns (as opposed to methods that learn from partial returns, considered in the next chapter).

Despite the differences between Monte Carlo and DP methods, the most important ideas carry over from DP to the Monte Carlo case. Not only do they compute the same value functions, but they interact to attain optimality in essentially the same way. As

in the DP chapter, we consider first policy evaluation, the computation of V^π and Q^π for a fixed arbitrary policy π , then policy improvement, and, finally, generalized policy iteration. Each of these ideas taken from DP is extended to the Monte Carlo case in which only sample experience is available.

5.1 Monte Carlo Policy Evaluation

We begin by considering Monte Carlo methods for learning the state-value function for a given policy. Recall that the value of a state is the expected return—expected cumulative future discounted reward—starting from that state. An obvious way to estimate it from experience, then, is simply to average the returns observed after visits to that state. As more returns are observed, the average should converge to the expected value. This idea underlies all Monte Carlo methods.

In particular, suppose we wish to estimate $V^\pi(s)$, the value of a state s under policy π , given a set of episodes obtained by following π and passing through s . Each occurrence of state s in an episode is called a *visit* to s . The *every-visit MC method* estimates $V^\pi(s)$ as the average of the returns following all the visits to s in a set of episodes. Within a given episode, the first time s is visited is called the *first visit* to s . The *first-visit MC method* averages just the returns following first visits to s . These two Monte Carlo methods are very similar but have slightly different theoretical properties. First-visit MC has been most widely studied, dating back to the 1940s, and is the one we focus on in this chapter. We reconsider every-visit MC in Chapter 7. First-visit MC is shown in procedural form in Figure 5.1.

Both first-visit MC and every-visit MC converge to $V^\pi(s)$ as the number of visits (or first visits) to s goes to infinity. This is easy to see for the case of first-visit MC. In this case each return is an independent, identically distributed estimate of $V^\pi(s)$. By the law of large numbers the sequence of averages of these estimates converges to their expected value. Each average is itself an unbiased estimate, and the standard deviation of its error falls as $1/\sqrt{n}$, where n is the number of returns averaged. Every-visit MC is less straightforward, but its estimates also converge asymptotically to $V^\pi(s)$.

The use of Monte Carlo methods is best illustrated through an example.

Example 5.1 *Blackjack* is a popular casino card game. The object is to obtain cards the sum of whose numerical values is as great as possible without exceeding 21. All face cards count as 10, and the ace can count either as 1 or as 11. We consider the version in which each player competes independently against the dealer. The game

```

Initialize:
     $\pi \leftarrow$  policy to be evaluated
     $V \leftarrow$  an arbitrary state-value function
     $Returns(s) \leftarrow$  an empty list, for all  $s \in \mathcal{S}$ 

Repeat forever:
    (a) Generate an episode using  $\pi$ 
    (b) For each state  $s$  appearing in the episode:
         $R \leftarrow$  return following the first occurrence of  $s$ 
        Append  $R$  to  $Returns(s)$ 
         $V(s) \leftarrow \text{average}(Returns(s))$ 

```

Figure 5.1 First-visit MC method for estimating V^π .

begins with two cards dealt to both dealer and player. One of the dealer's cards is faceup and the other is facedown. If the player has 21 immediately (an ace and a 10-card), it is called a *natural*. He then wins unless the dealer also has a natural, in which case the game is a draw. If the player does not have a natural, then he can request additional cards, one by one (*hits*), until he either stops (*sticks*) or exceeds 21 (*goes bust*). If he goes bust, he loses; if he sticks, then it becomes the dealer's turn. The dealer hits or sticks according to a fixed strategy without choice: he sticks on any sum of 17 or greater, and hits otherwise. If the dealer goes bust, then the player wins; otherwise, the outcome—win, lose, or draw—is determined by whose final sum is closer to 21.

Playing blackjack is naturally formulated as an episodic finite MDP. Each game of blackjack is an episode. Rewards of +1, −1, and 0 are given for winning, losing, and drawing, respectively. All rewards within a game are zero, and we do not discount ($\gamma = 1$); therefore these terminal rewards are also the returns. The player's actions are to hit or to stick. The states depend on the player's cards and the dealer's showing card. We assume that cards are dealt from an infinite deck (i.e., with replacement) so that there is no advantage to keeping track of the cards already dealt. If the player holds an ace that he could count as 11 without going bust, then the ace is said to be *usable*. In this case it is always counted as 11 because counting it as 1 would make the sum 11 or less, in which case there is no decision to be made because, obviously, the player should always hit. Thus, the player makes decisions on the basis of three variables: his current sum (12–21), the dealer's one showing card (ace–10), and whether or not he holds a usable ace. This makes for a total of 200 states.

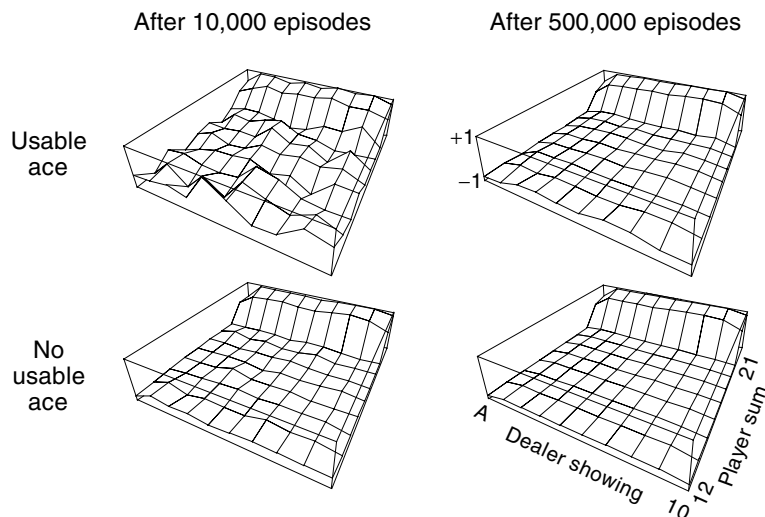


Figure 5.2 Approximate state-value functions for the blackjack policy that sticks only on 20 or 21, computed by Monte Carlo policy evaluation.

Consider the policy that sticks if the player's sum is 20 or 21, and otherwise hits. To find the state-value function for this policy by a Monte Carlo approach, one simulates many blackjack games using the policy and averages the returns following each state. Note that in this task the same state never recurs within one episode, so there is no difference between first-visit and every-visit MC methods. In this way, we obtained the estimates of the state-value function shown in Figure 5.2. The estimates for states with a usable ace are less certain and less regular because these states are less common. In any event, after 500,000 games the value function is very well approximated.

Although we have complete knowledge of the environment in this task, it would not be easy to apply DP policy evaluation to compute the value function. DP methods require the distribution of next events—in particular, they require the quantities $\mathcal{P}_{ss'}^a$ and $\mathcal{R}_{ss'}^a$ —and it is not easy to determine these for blackjack. For example, suppose the player's sum is 14 and he chooses to stick. What is his expected reward as a function of the dealer's showing card? All of these expected rewards and transition probabilities must be computed *before* DP can be applied, and such computations are often complex and error-prone. In contrast, generating the sample games required by Monte Carlo methods is easy. This is the case surprisingly often; the ability of Monte Carlo methods to work with sample episodes alone can be a significant advantage even when one has complete knowledge of the environment's dynamics. ■

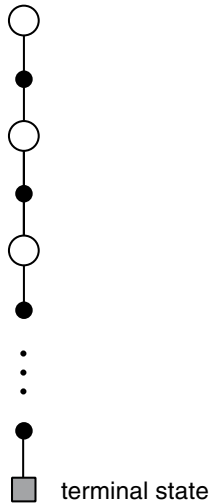


Figure 5.3 The backup diagram for Monte Carlo estimation of V^π .

Can we generalize the idea of backup diagrams to Monte Carlo algorithms? The general idea of a backup diagram is to show at the top the root node to be updated and to show below all the transitions and leaf nodes whose rewards and estimated values contribute to the update. For Monte Carlo estimation of V^π , the root is a state node, and below is the entire sequence of transitions along a particular episode, ending at the terminal state, as in Figure 5.3. Whereas the DP diagram (Figure 3.4a) shows all possible transitions, the Monte Carlo diagram shows only those sampled on the one episode. Whereas the DP diagram includes only one-step transitions, the Monte Carlo diagram goes all the way to the end of the episode. These differences in the diagrams accurately reflect the fundamental differences between the algorithms.

An important fact about Monte Carlo methods is that the estimates for each state are independent. The estimate for one state does not build upon the estimate of any other state, as is the case in DP. In other words, Monte Carlo methods do not “bootstrap” as we described it in the previous chapter.

In particular, note that the computational expense of estimating the value of a single state is independent of the number of states. This can make Monte Carlo methods particularly attractive when one requires the value of only a subset of the states. One can generate many sample episodes starting from these states, averaging returns only from these states, ignoring all others. This is a third advantage Monte Carlo methods can have over DP methods (after the ability to learn from actual experience and from simulated experience).

Example 5.2: Soap Bubble Suppose a wire frame forming a closed loop is dunked in soapy water to form a soap surface or bubble conforming at its edges to the wire frame. If the geometry of the wire frame is irregular but known, how can you compute the shape of the surface? The shape has the property that the total force on each point exerted by neighboring points is zero (or else the shape would change). This means that the surface's height at any point is the average of its heights at points in a small circle around that point. In addition, the surface must meet at its boundaries with the wire frame. The usual approach to problems of this kind is to put a grid over the area covered by the surface and solve for its height at the grid points by an iterative computation. Grid points at the boundary are forced to the wire frame, and all others are adjusted toward the average of the heights of their four nearest neighbors. This process then iterates, much like DP's iterative policy evaluation, and ultimately converges to a close approximation to the desired surface.

This is similar to the kind of problem for which Monte Carlo methods were originally designed. Instead of the iterative computation described above, imagine standing on the surface and taking a random walk, stepping randomly from grid point to neighboring grid point, with equal probability, until you reach the boundary. It turns out that the expected value of the height at the boundary is a close approximation to the height of the desired surface at the starting point (in fact, it is exactly the value computed by the iterative method described above). Thus, one can closely approximate the height of the surface at a point by simply averaging the boundary heights of many walks started at the point. If one is interested in only the value at one point, or any fixed small set of points, then this Monte Carlo method can be far more efficient than the iterative method based on local consistency. ■

Exercise 5.1 Consider the diagrams on the right in Figure 5.2. Why does the value function jump up for the last two rows in the rear? Why does it drop off for the whole last row on the left? Why are the frontmost values higher in the upper diagrams than in the lower?

5.2 Monte Carlo Estimation of Action Values

If a model is not available, then it is particularly useful to estimate *action* values rather than *state* values. With a model, state values alone are sufficient to determine a policy; one simply looks ahead one step and chooses whichever action leads to the best combination of reward and next state, as we did in the chapter on DP. Without a

model, however, state values alone are not sufficient. One must explicitly estimate the value of each action in order for the values to be useful in suggesting a policy. Thus, one of our primary goals for Monte Carlo methods is to estimate Q^* . To achieve this, we first consider another policy evaluation problem.

The policy evaluation problem for action values is to estimate $Q^\pi(s, a)$, the expected return when starting in state s , taking action a , and thereafter following policy π . The Monte Carlo methods here are essentially the same as just presented for state values. The every-visit MC method estimates the value of a state–action pair as the average of the returns that have followed visits to the state in which the action was selected. The first-visit MC method averages the returns following the first time in each episode that the state was visited and the action was selected. These methods converge quadratically, as before, to the true expected values as the number of visits to each state–action pair approaches infinity.

The only complication is that many relevant state–action pairs may never be visited. If π is a deterministic policy, then in following π one will observe returns only for one of the actions from each state. With no returns to average, the Monte Carlo estimates of the other actions will not improve with experience. This is a serious problem because the purpose of learning action values is to help in choosing among the actions available in each state. To compare alternatives we need to estimate the value of *all* the actions from each state, not just the one we currently favor.

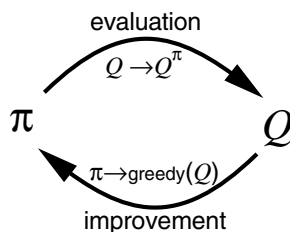
This is the general problem of *maintaining exploration*, as discussed in the context of the n -armed bandit problem in Chapter 2. For policy evaluation to work for action values, we must assure continual exploration. One way to do this is by specifying that the first step of each episode starts at a state–action *pair*, and that every such pair has a nonzero probability of being selected as the start. This guarantees that all state–action pairs will be visited an infinite number of times in the limit of an infinite number of episodes. We call this the assumption of *exploring starts*.

The assumption of exploring starts is sometimes useful, but of course it cannot be relied upon in general, particularly when learning directly from real interactions with an environment. In that case the starting conditions are unlikely to be so helpful. The most common alternative approach to assuring that all state–action pairs are encountered is to consider only policies that are stochastic with a nonzero probability of selecting all actions. We discuss two important variants of this approach in later sections. For now, we retain the assumption of exploring starts and complete the presentation of a full Monte Carlo control method.

Exercise 5.2 What is the backup diagram for Monte Carlo estimation of Q^π ?

5.3 Monte Carlo Control

We are now ready to consider how Monte Carlo estimation can be used in control, that is, to approximate optimal policies. The overall idea is to proceed according to the same pattern as in the DP chapter, that is, according to the idea of generalized policy iteration (GPI). In GPI one maintains both an approximate policy and an approximate value function. The value function is repeatedly altered to more closely approximate the value function for the current policy, and the policy is repeatedly improved with respect to the current value function:



These two kinds of changes work against each other to some extent, as each creates a moving target for the other, but together they cause both policy and value function to approach optimality.

To begin, let us consider a Monte Carlo version of classical policy iteration. In this method, we perform alternating complete steps of policy evaluation and policy improvement, beginning with an arbitrary policy π_0 and ending with the optimal policy and optimal action-value function:

$$\pi_0 \xrightarrow{E} Q^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} Q^{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi^* \xrightarrow{E} Q^*,$$

where $\xrightarrow{E}$ denotes a complete policy evaluation and $\xrightarrow{I}$ denotes a complete policy improvement. Policy evaluation is done exactly as described in the preceding section. Many episodes are experienced, with the approximate action-value function approaching the true function asymptotically. For the moment, let us assume that we do indeed observe an infinite number of episodes and that, in addition, the episodes are generated with exploring starts. Under these assumptions, the Monte Carlo methods will compute each Q^{π_k} exactly, for arbitrary π_k .

Policy improvement is done by making the policy greedy with respect to the current value function. In this case we have an *action*-value function, and therefore no model is needed to construct the greedy policy. For any action-value function Q , the

corresponding greedy policy is the one that, for each $s \in \mathcal{S}$, deterministically chooses an action with maximal Q -value:

$$\pi(s) = \arg \max_a Q(s, a). \quad (5.1)$$

Policy improvement then can be done by constructing each π_{k+1} as the greedy policy with respect to Q^{π_k} . The policy improvement theorem (Section 4.2) then applies to π_k and π_{k+1} because, for all $s \in \mathcal{S}$,

$$\begin{aligned} Q^{\pi_k}(s, \pi_{k+1}(s)) &= Q^{\pi_k}(s, \arg \max_a Q^{\pi_k}(s, a)) \\ &= \max_a Q^{\pi_k}(s, a) \\ &\geq Q^{\pi_k}(s, \pi_k(s)) \\ &= V^{\pi_k}(s). \end{aligned}$$

As we discussed in the previous chapter, the theorem assures us that each π_{k+1} is uniformly better than π_k , unless it is equal to π_k , in which case they are both optimal policies. This in turn assures us that the overall process converges to an optimal policy and the optimal value function. In this way Monte Carlo methods can be used to find optimal policies given only sample episodes and no other knowledge of the environment's dynamics.

We made two unlikely assumptions above in order to easily obtain this guarantee of convergence for the Monte Carlo method. One was that the episodes have exploring starts, and the other was that policy evaluation could be done with an infinite number of episodes. To obtain a practical algorithm we will have to remove both assumptions. We postpone consideration of the first assumption until later in this chapter.

For now we focus on the assumption that policy evaluation operates on an infinite number of episodes. This assumption is relatively easy to remove. In fact, the same issue arises even in classical DP methods such as iterative policy evaluation, which also converge only asymptotically to the true value function. In both DP and Monte Carlo cases there are two ways to solve the problem. One is to hold firm to the idea of approximating Q^{π_k} in each policy evaluation. Measurements and assumptions are made to obtain bounds on the magnitude and probability of error in the estimates, and then sufficient steps are taken during each policy evaluation to assure that these bounds are sufficiently small. This approach can probably be made completely satisfactory in the sense of guaranteeing correct convergence up to some level of approximation. However, it is also likely to require far too many episodes to be useful in practice on any but the smallest problems.

The second approach to avoiding the infinite number of episodes nominally required for policy evaluation is to forgo trying to complete policy evaluation before returning to policy improvement. On each evaluation step we move the value function toward Q^{π_k} , but we do not expect to actually get close except over many steps. We used this idea when we first introduced the idea of GPI in Section 4.6. One extreme form of the idea is value iteration, in which only one iteration of iterative policy evaluation is performed between each step of policy improvement. The in-place version of value iteration is even more extreme; there we alternate between improvement and evaluation steps for single states.

For Monte Carlo policy evaluation it is natural to alternate between evaluation and improvement on an episode-by-episode basis. After each episode, the observed returns are used for policy evaluation, and then the policy is improved at all the states visited in the episode. A complete simple algorithm along these lines is given in Figure 5.4. We call this algorithm *Monte Carlo ES*, for Monte Carlo with Exploring Starts.

In Monte Carlo ES, all the returns for each state–action pair are accumulated and averaged, irrespective of what policy was in force when they were observed. It is easy to see that Monte Carlo ES cannot converge to any suboptimal policy. If it did, then the value function would eventually converge to the value function for that policy, and that in turn would cause the policy to change. Stability is achieved only

```

Initialize, for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ :
     $Q(s, a) \leftarrow$  arbitrary
     $\pi(s) \leftarrow$  arbitrary
     $Returns(s, a) \leftarrow$  empty list

Repeat forever:
    (a) Generate an episode using exploring starts and  $\pi$ 
    (b) For each pair  $s, a$  appearing in the episode:
         $R \leftarrow$  return following the first occurrence of  $s, a$ 
        Append  $R$  to  $Returns(s, a)$ 
         $Q(s, a) \leftarrow \text{average}(Returns(s, a))$ 
    (c) For each  $s$  in the episode:
         $\pi(s) \leftarrow \arg \max_a Q(s, a)$ 

```

Figure 5.4 Monte Carlo ES: A Monte Carlo control algorithm assuming exploring starts.

when both the policy and the value function are optimal. Convergence to this optimal fixed point seems inevitable as the changes to the action-value function decrease over time, but has not yet been formally proved. In our opinion, this is one of the most fundamental open questions in reinforcement learning.

Example 5.3: Solving Blackjack It is straightforward to apply Monte Carlo ES to blackjack. Since the episodes are all simulated games, it is easy to arrange for exploring starts that include all possibilities. In this case one simply picks the dealer’s cards, the player’s sum, and whether or not the player has a usable ace, all at random with equal probability. As the initial policy we use the policy evaluated in the previous blackjack example, that which sticks only on 20 or 21. The initial action-value function can be zero for all state–action pairs. Figure 5.5 shows the optimal policy for blackjack found by Monte Carlo ES. This policy is the same as the “basic” strategy of Thorp (1966) with the sole exception of the leftmost notch in the policy for a usable ace, which is not present in Thorp’s strategy. We are uncertain of the reason for this discrepancy, but confident that what is shown here is indeed the optimal policy for the version of blackjack we have described. ■

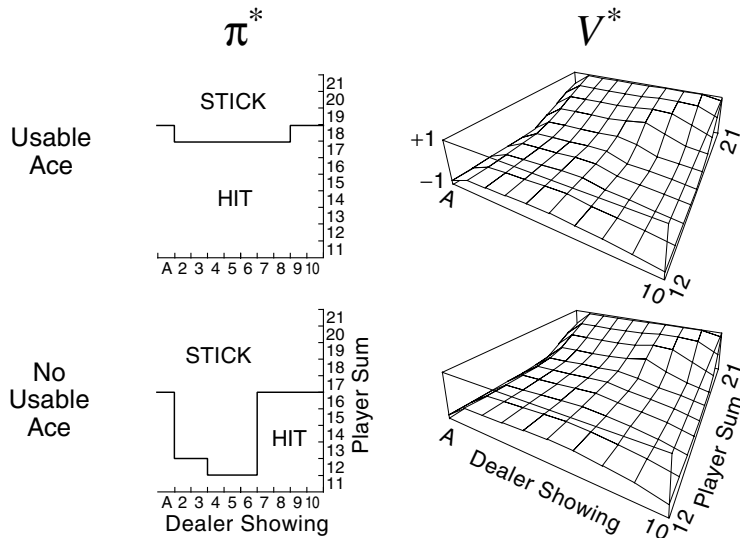


Figure 5.5 The optimal policy and state-value function for blackjack, found by Monte Carlo ES (Figure 5.4). The state-value function shown was computed from the action-value function found by Monte Carlo ES.

5.4 On-Policy Monte Carlo Control

How can we avoid the unlikely assumption of exploring starts? The only general way to ensure that all actions are selected infinitely often is for the agent to continue to select them. There are two approaches to ensuring this, resulting in what we call *on-policy* methods and *off-policy* methods. On-policy methods attempt to evaluate or improve the policy that is used to make decisions. In this section we present an on-policy Monte Carlo control method in order to illustrate the idea.

In on-policy control methods the policy is generally *soft*, meaning that $\pi(s, a) > 0$ for all $s \in \mathcal{S}$ and all $a \in \mathcal{A}(s)$. There are many possible variations on on-policy methods. One possibility is to gradually shift the policy toward a deterministic optimal policy. Many of the methods discussed in Chapter 2 provide mechanisms for this. The on-policy method we present in this section uses ϵ -greedy policies, meaning that most of the time they choose an action that has maximal estimated action value, but with probability ϵ they instead select an action at random. That is, all nongreedy actions are given the minimal probability of selection, $\frac{\epsilon}{|\mathcal{A}(s)|}$, and the remaining bulk of the probability, $1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|}$, is given to the greedy action. The ϵ -greedy policies are examples of ϵ -soft policies, defined as policies for which $\pi(s, a) \geq \frac{\epsilon}{|\mathcal{A}(s)|}$ for all states and actions, for some $\epsilon > 0$. Among ϵ -soft policies, ϵ -greedy policies are in some sense those that are closest to greedy.

The overall idea of on-policy Monte Carlo control is still that of GPI. As in Monte Carlo ES, we use first-visit MC methods to estimate the action-value function for the current policy. Without the assumption of exploring starts, however, we cannot simply improve the policy by making it greedy with respect to the current value function, because that would prevent further exploration of nongreedy actions. Fortunately, GPI does not require that the policy be taken all the way to a greedy policy, only that it be moved *toward* a greedy policy. In our on-policy method we will move it only to an ϵ -greedy policy. For any ϵ -soft policy, π , any ϵ -greedy policy with respect to Q^π is guaranteed to be better than or equal to π .

That any ϵ -greedy policy with respect to Q^π is an improvement over any ϵ -soft policy π is assured by the policy improvement theorem. Let π' be the ϵ -greedy policy. The conditions of the policy improvement theorem apply because for any $s \in \mathcal{S}$:

$$\begin{aligned}
Q^\pi(s, \pi'(s)) &= \sum_a \pi'(s, a) Q^\pi(s, a) \\
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s, a) + (1 - \epsilon) \max_a Q^\pi(s, a) \\
&\geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s, a) + (1 - \epsilon) \sum_a \frac{\pi(s, a) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1 - \epsilon} Q^\pi(s, a)
\end{aligned} \tag{5.2}$$

(the sum is a weighted average with nonnegative weights summing to 1, and as such it must be less than or equal to the largest number averaged)

$$\begin{aligned}
&= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s, a) - \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s, a) + \sum_a \pi(s, a) Q^\pi(s, a) \\
&= V^\pi(s).
\end{aligned}$$

Thus, by the policy improvement theorem, $\pi' \geq \pi$ (i.e., $V^{\pi'}(s) \geq V^\pi(s)$, for all $s \in \mathcal{S}$). We now prove that equality can hold only when both π' and π are optimal among the ϵ -soft policies, that is, when they are better than or equal to all other ϵ -soft policies.

Consider a new environment that is just like the original environment, except with the requirement that policies be ϵ -soft “moved inside” the environment. The new environment has the same action and state sets as the original and behaves as follows. If in state s and taking action a , then with probability $1 - \epsilon$ the new environment behaves exactly like the old environment. With probability ϵ it repicks the action at random, with equal probabilities, and then behaves like the old environment with the new, random action. The best one can do in this new environment with general policies is the same as the best one could do in the original environment with ϵ -soft policies. Let $\tilde{V}^*$ denote the optimal value function for the new environment. Then a policy π is optimal among ϵ -soft policies if and only if $V^\pi = \tilde{V}^*$. From the definition of $\tilde{V}^*$ we know that it is the unique solution to

$$\begin{aligned}
\tilde{V}^*(s) &= (1 - \epsilon) \max_a \sum_{s'} \mathcal{P}_{ss'}^a [\mathcal{R}_{ss'}^a + \gamma \tilde{V}^*(s')] \\
&\quad + \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a \sum_{s'} \mathcal{P}_{ss'}^a [\mathcal{R}_{ss'}^a + \gamma \tilde{V}^*(s')].
\end{aligned}$$

When equality holds and the ϵ -soft policy π is no longer improved, then we also know, from (5.2), that

$$\begin{aligned} V^\pi(s) &= (1 - \epsilon) \max_a Q^\pi(s, a) + \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a Q^\pi(s, a) \\ &= (1 - \epsilon) \max_a \sum_{s'} \mathcal{P}_{ss'}^a [\mathcal{R}_{ss'}^a + \gamma V^\pi(s')] \\ &\quad + \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a \sum_{s'} \mathcal{P}_{ss'}^a [\mathcal{R}_{ss'}^a + \gamma V^\pi(s')]. \end{aligned}$$

However, this equation is the same as the previous one, except for the substitution of V^π for $\tilde{V}^*$. Since $\tilde{V}^*$ is the unique solution, it must be that $V^\pi = \tilde{V}^*$.

In essence, we have shown in the last few pages that policy iteration works for ϵ -soft policies. Using the natural notion of greedy policy for ϵ -soft policies, one is assured of improvement on every step, except when the best policy has been found among the ϵ -soft policies. This analysis is independent of how the action-value functions are determined at each stage, but it does assume that they are computed exactly. This brings us to roughly the same point as in the previous section. Now we only achieve the best policy among the ϵ -soft policies, but on the other hand, we have eliminated the assumption of exploring starts. The complete algorithm is given in Figure 5.6.

5.5 Evaluating One Policy While Following Another

So far we have considered methods for estimating the value functions for a policy given an infinite supply of episodes generated using that policy. Suppose now that all we have are episodes generated from a *different* policy. That is, suppose we wish to estimate V^π or Q^π , but all we have are episodes following π' , where $\pi' \neq \pi$. Can we learn the value function for a policy given only experience “off” the policy?

Happily, in many cases we can. Of course, in order to use episodes from π' to estimate values for π , we require that every action taken under π is also taken, at least occasionally, under π' . That is, we require that $\pi(s, a) > 0$ implies $\pi'(s, a) > 0$. In the episodes generated using π' , consider the i th first visit to state s and the complete sequence of states and actions following that visit. Let $p_i(s)$ and $p'_i(s)$ denote the probabilities of that complete sequence happening given policies π and π' and starting from s . Let $R_i(s)$ denote the corresponding observed return from state s .

```

Initialize, for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$ :
     $Q(s, a) \leftarrow$  arbitrary
     $Returns(s, a) \leftarrow$  empty list
     $\pi \leftarrow$  an arbitrary  $\epsilon$ -soft policy

Repeat forever:
    (a) Generate an episode using  $\pi$ 
    (b) For each pair  $s, a$  appearing in the episode:
         $R \leftarrow$  return following the first occurrence of  $s, a$ 
        Append  $R$  to  $Returns(s, a)$ 
         $Q(s, a) \leftarrow \text{average}(Returns(s, a))$ 
    (c) For each  $s$  in the episode:
         $a^* \leftarrow \arg \max_a Q(s, a)$ 
        For all  $a \in \mathcal{A}(s)$ :
             $\pi(s, a) \leftarrow \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(s)| & \text{if } a = a^* \\ \epsilon/|\mathcal{A}(s)| & \text{if } a \neq a^* \end{cases}$ 

```

Figure 5.6 An ϵ -soft on-policy Monte Carlo control algorithm.

To average these to obtain an unbiased estimate of $V^\pi(s)$, we need only weight each return by its relative probability of occurring under π and π' , that is, by $p_i(s)/p'_i(s)$. The desired Monte Carlo estimate after observing n_s returns from state s is then

$$V(s) = \frac{\sum_{i=1}^{n_s} \frac{p_i(s)}{p'_i(s)} R_i(s)}{\sum_{i=1}^{n_s} \frac{p_i(s)}{p'_i(s)}}. \quad (5.3)$$

This equation involves the probabilities $p_i(s)$ and $p'_i(s)$, which are normally considered unknown in applications of Monte Carlo methods. Fortunately, here we need only their ratio, $p_i(s)/p'_i(s)$, which *can* be determined with no knowledge of the environment's dynamics. Let $T_i(s)$ be the time of termination of the i^{th} episode involving state s . Then

$$p_i(s) = \prod_{k=t}^{T_i(s)-1} \pi(s_k, a_k) \mathcal{P}_{s_k s_{k+1}}^{a_k}$$

and

$$\frac{p_i(s_t)}{p'_i(s_t)} = \frac{\prod_{k=t}^{T_i(s)-1} \pi(s_k, a_k) \mathcal{P}_{s_k s_{k+1}}^{a_k}}{\prod_{k=t}^{T_i(s)-1} \pi'(s_k, a_k) \mathcal{P}_{s_k s_{k+1}}^{a_k}} = \prod_{k=t}^{T_i(s)-1} \frac{\pi(s_k, a_k)}{\pi'(s_k, a_k)}.$$

Thus the weight needed in (5.3), $p_i(s)/p'_i(s)$, depends only on the two policies and not at all on the environment's dynamics.

Exercise 5.3 What is the Monte Carlo estimate analogous to (5.3) for *action* values, given returns generated using π' ?

5.6 Off-Policy Monte Carlo Control

We are now ready to present an example of the second class of learning control methods we consider in this book: off-policy methods. Recall that the distinguishing feature of on-policy methods is that they estimate the value of a policy while using it for control. In off-policy methods these two functions are separated. The policy used to generate behavior, called the *behavior* policy, may in fact be unrelated to the policy that is evaluated and improved, called the *estimation* policy. An advantage of this separation is that the estimation policy may be deterministic (e.g., greedy), while the behavior policy can continue to sample all possible actions.

Off-policy Monte Carlo control methods use the technique presented in the preceding section for estimating the value function for one policy while following another. They follow the behavior policy while learning about and improving the estimation policy. This technique requires that the behavior policy have a nonzero probability of selecting all actions that might be selected by the estimation policy. To explore all possibilities, we require that the behavior policy be soft.

Figure 5.7 shows an off-policy Monte Carlo method, based on GPI, for computing Q^* . The behavior policy π' is maintained as an arbitrary soft policy. The estimation policy π is the greedy policy with respect to Q , an estimate of Q^π . The behavior policy chosen in (a) can be anything, but in order to assure convergence of π to the optimal policy, an infinite number of returns suitable for use in (c) must be obtained for each pair of state and action. This can be assured by careful choice of the behavior policy. For example, any ϵ -soft behavior policy will suffice.

A potential problem is that this method learns only from the *tails* of episodes, after the last nongreedy action. If nongreedy actions are frequent, then learning will be slow, particularly for states appearing in the early portions of long episodes. Potentially, this could greatly slow learning. There has been insufficient experience with off-policy Monte Carlo methods to assess how serious this problem is.


```

Initialize, for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$ :
     $Q(s, a) \leftarrow$  arbitrary
     $N(s, a) \leftarrow 0$  ; Numerator and
     $D(s, a) \leftarrow 0$  ; Denominator of  $Q(s, a)$ 
     $\pi \leftarrow$  an arbitrary deterministic policy

Repeat forever:
    (a) Select a policy  $\pi'$  and use it to generate an episode:
         $s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T, s_T$ 
    (b)  $\tau \leftarrow$  latest time at which  $a_\tau \neq \pi(s_\tau)$ 
    (c) For each pair  $s, a$  appearing in the episode at time  $\tau$  or later:
         $t \leftarrow$  the time of first occurrence of  $s, a$  such that  $t \geq \tau$ 
         $w \leftarrow \prod_{k=t+1}^{T-1} \frac{1}{\pi'(s_k, a_k)}$ 
         $N(s, a) \leftarrow N(s, a) + w R_t$ 
         $D(s, a) \leftarrow D(s, a) + w$ 
         $Q(s, a) \leftarrow \frac{N(s, a)}{D(s, a)}$ 
    (d) For each  $s \in \mathcal{S}$ :
         $\pi(s) \leftarrow \arg \max_a Q(s, a)$ 

```

Figure 5.7 An off-policy Monte Carlo control algorithm.

Exercise 5.4: Racetrack (programming) Consider driving a race car around a turn like those shown in Figure 5.8. You want to go as fast as possible, but not so fast as to run off the track. In our simplified racetrack, the car is at one of a discrete set of grid positions, the cells in the diagram. The velocity is also discrete, a number of grid cells moved horizontally and vertically per time step. The actions are increments to the velocity components. Each may be changed by $+1$, -1 , or 0 in one step, for a total of nine actions. Both velocity components are restricted to be nonnegative and less than 5, and they cannot both be zero. Each episode begins in one of the randomly selected start states and ends when the car crosses the finish line. The rewards are -1 for each step that stays on the track, and -5 if the agent tries to drive off the track. Actually leaving the track is not allowed, but the position is always advanced by at least one cell along either the horizontal or vertical axes. With these restrictions and considering only right turns, such as shown in the figure, all episodes are guaranteed to terminate, yet the optimal policy is unlikely to be excluded. To make the task more challenging, we assume that on half of the time steps the position is displaced forward

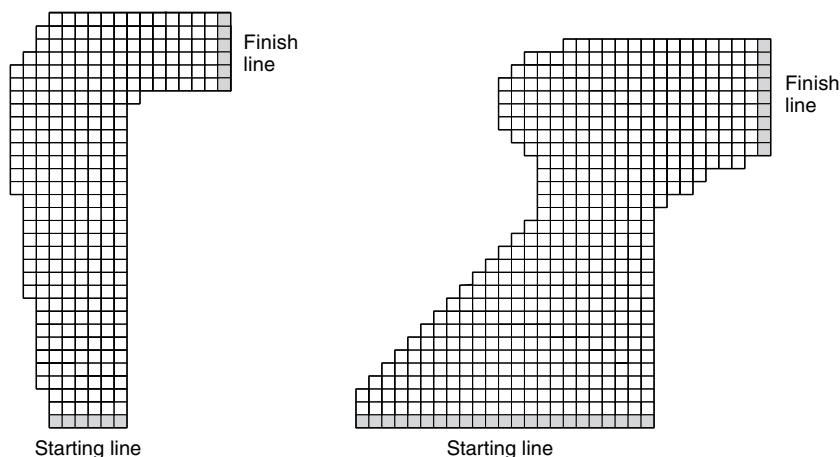


Figure 5.8 A couple of right turns for the racetrack task.

or to the right by one additional cell beyond that specified by the velocity. Apply the on-policy Monte Carlo control method to this task to compute the optimal policy from each starting state. Exhibit several trajectories following the optimal policy.

5.7 Incremental Implementation

Monte Carlo methods can be implemented incrementally, on an episode-by-episode basis, using extensions of techniques described in Chapter 2. They use averages of *returns* just as some of the methods for solving n -armed bandit tasks described in Chapter 2 use averages of *rewards*. The techniques in Sections 2.5 and 2.6 extend immediately to the Monte Carlo case. They enable Monte Carlo methods to process each new return incrementally with no increase in computation or memory as the number of episodes increases.

There are two differences between the Monte Carlo and bandit cases. One is that the Monte Carlo case typically involves multiple situations, that is, a different averaging process for each state, whereas bandit problems involve just one state (at least in the simple form treated in Chapter 2). The other difference is that the reward distributions in bandit problems are typically stationary, whereas in Monte Carlo methods the return distributions are typically nonstationary. This is because the returns depend on the policy, and the policy is typically changing and improving over time.

The incremental implementation described in Section 2.5 handles the case of simple or arithmetic averages, in which each return is weighted equally. Suppose we instead want to implement a *weighted* average, in which each return R_n is weighted by w_n , and we want to compute

$$V_n = \frac{\sum_{k=1}^n w_k R_k}{\sum_{k=1}^n w_k}. \quad (5.4)$$

For example, the method described for estimating one policy while following another in Section 5.5 uses weights of $w_n(s) = p_n(s)/p'_n(s)$. Weighted averages also have a simple incremental update rule. In addition to keeping track of V_n , we must maintain for each state the cumulative sum W_n of the weights given to the first n returns. The update rule for V_n is

$$V_{n+1} = V_n + \frac{w_{n+1}}{W_{n+1}} [R_{n+1} - V_n] \quad (5.5)$$

and

$$W_{n+1} = W_n + w_{n+1}$$

where $V_0 = W_0 = 0$.

Exercise 5.5 Modify the algorithm for first-visit MC policy evaluation (Figure 5.1) to use the incremental implementation for stationary averages described in Section 2.5.

Exercise 5.6 Derive the weighted-average update rule (5.5) from (5.4). Follow the pattern of the derivation of the unweighted rule (2.4) from (2.1).

Exercise 5.7 Modify the algorithm for the off-policy Monte Carlo control algorithm (Figure 5.7) to use the method described above for incrementally computing weighted averages.

5.8 Summary

The Monte Carlo methods presented in this chapter learn value functions and optimal policies from experience in the form of *sample episodes*. This gives them at least three kinds of advantages over DP methods. First, they can be used to learn optimal behavior directly from interaction with the environment, with no model of the environment's dynamics. Second, they can be used with simulation or *sample models*. For surprisingly many applications it is easy to simulate sample episodes even

though it is difficult to construct the kind of explicit model of transition probabilities required by DP methods. Third, it is easy and efficient to *focus* Monte Carlo methods on a small subset of the states. A region of special interest can be accurately evaluated without going to the expense of accurately evaluating the rest of the state set (we explore this further in Chapter 9).

A fourth advantage of Monte Carlo methods, which we discuss later in the book, is that they may be less harmed by violations of the Markov property. This is because they do not update their value estimates on the basis of the value estimates of successor states. In other words, it is because they do not bootstrap.

In designing Monte Carlo control methods we have followed the overall schema of *generalized policy iteration* (GPI) introduced in Chapter 4. GPI involves interacting processes of policy evaluation and policy improvement. Monte Carlo methods provide an alternative policy evaluation process. Rather than use a model to compute the value of each state, they simply average many returns that start in the state. Because a state's value is the expected return, this average can become a good approximation to the value. In control methods we are particularly interested in approximating action-value functions, because these can be used to improve the policy without requiring a model of the environment's transition dynamics. Monte Carlo methods intermix policy evaluation and policy improvement steps on an episode-by-episode basis, and can be incrementally implemented on an episode-by-episode basis.

Maintaining *sufficient exploration* is an issue in Monte Carlo control methods. It is not enough just to select the actions currently estimated to be best, because then no returns will be obtained for alternative actions, and it may never be learned that they are actually better. One approach is to ignore this problem by assuming that episodes begin with state–action pairs randomly selected to cover all possibilities. Such *exploring starts* can sometimes be arranged in applications with simulated episodes, but are unlikely in learning from real experience. Instead, one of two general approaches can be used. In *on-policy* methods, the agent commits to always exploring and tries to find the best policy that still explores. In *off-policy* methods, the agent also explores, but learns a deterministic optimal policy that may be unrelated to the policy followed. More instances of both kinds of methods are presented in the next chapter.

Monte Carlo methods for reinforcement learning have been explicitly identified only recently. Their convergence properties are not yet clear, and their effectiveness in practice has been little tested. At present, their primary significance is their simplicity and their relationships to other methods.

Monte Carlo methods differ from DP methods in two ways. First, they operate on sample experience, and thus can be used for direct learning without a model.

Second, they do not bootstrap. That is, they do not update their value estimates on the basis of other value estimates. These two differences are not tightly linked and can be separated. In the next chapter we consider methods that learn from experience, like Monte Carlo methods, but also bootstrap, like DP methods.

5.9 Bibliographical and Historical Remarks

The term “Monte Carlo” dates from the 1940s, when physicists at Los Alamos devised games of chance that they could study to help understand complex physical phenomena relating to the atom bomb. Coverage of Monte Carlo methods in this sense can be found in several textbooks (e.g., Kalos and Whitlock, 1986; Rubinstein, 1981).

An early use of Monte Carlo methods to estimate action values in a reinforcement learning context was by Michie and Chambers (1968). In pole balancing (Example 3.4), they used averages of episode durations to assess the worth (expected balancing “life”) of each possible action in each state, and then used these assessments to control action selections. Their method is similar in spirit to Monte Carlo ES. In our terms, they used a form of every-visit MC method. Narendra and Wheeler (1986) studied a Monte Carlo method for ergodic finite Markov chains that used the return accumulated from one visit to a state to the next as a reward for adjusting a learning automaton’s action probabilities.

Barto and Duff (1994) discussed policy evaluation in the context of classical Monte Carlo algorithms for solving systems of linear equations. They used the analysis of Curtiss (1954) to point out the computational advantages of Monte Carlo policy evaluation for large problems. Singh and Sutton (1996) distinguished between every-visit and first-visit MC methods and proved results relating these methods to reinforcement learning algorithms.

The blackjack example is based on an example used by Widrow, Gupta, and Maitra (1973). The soap bubble example is a classical Dirichlet problem whose Monte Carlo solution was first proposed by Kakutani (1945; see Hersh and Griego, 1969; Doyle and Snell, 1984). The racetrack exercise is adapted from Barto, Bradtke, and Singh (1995), and from Gardner (1973).